PARCOURS PRATIQUE · ÉDITION 2026

Cours IA & OpenCode

Les 20 thématiques, résumées pour la présentation

Comprendre l'IA, installer OpenCode, choisir des modèles gratuits ou payants, créer des agents, des skills, des plugins et des serveurs MCP.

- DÉCOUVRIR
- INSTALLER
- PROVIDERS & MODÈLES
- LES AGENTS
- ÉTENDRE OPENCODE
- PRATIQUE & MÉTHODE

20 THÉMATIQUES · 6 MODULES · SUPPORT DE PRÉSENTATION

Le parcours en un coup d'œil

20 thématiques, 6 modules, un seul projet fil rouge : ton assistant de code.

Module 1 · Découvrir

01 Comprendre l'IA sans jargon

02 Le vocabulaire : modèle, provider, agent

Module 2 · Installer

03 Installer OpenCode (toutes les méthodes)

04 Premier lancement & l'interface

Module 3 · Providers & modèles

05 Qu'est-ce qu'un provider ?

06 Trouver des modèles gratuits

07 Choisir son modèle (gratuit ou payant)

08 Configurer ses providers (fichier de config)

09 Modèles locaux (Ollama, LM Studio, vLLM)

10 Réduire les coûts (et rester efficace)

Module 4 · Les agents

11 Qu'est-ce qu'un agent ?

12 Créer un agent local (pas à pas)

13 Trouver & intégrer des agents

Module 5 · Étendre OpenCode

14 Les Skills : des compétences réutilisables

15 Les plugins : brancher des outils

16 Les commandes : tes raccourcis

17 Les serveurs MCP : connecter le monde

Module 6 · Pratique & méthode

18 Bien prompter & le fichier AGENTS.md

19 Mini-projet : ton assistant de A à Z

20 Synthèse, dépannage & ressources

Comprendre l'IA sans jargon

« Les premières pendules molles caressées par l'aube. »

Avant d'installer quoi que ce soit, on pose les bases. En 10 minutes, tu vas comprendre ce qu'est réellement une « IA » aujourd'hui, pourquoi elle se trompe parfois, et surtout à quoi elle sert concrètement à un développeur.

AU PROGRAMME

- Qu'est-ce qu'une IA, vraiment ?
- Pourquoi ça se trompe parfois ? (les hallucinations)
- Ce que l'IA fait très bien (et ce qu'elle fait mal)
- À quoi ça sert pour un développeur ?
- Ce qu'on va construire ensemble
- Comment lire ce cours

À RETENIR

- ✓ Un « IA » de développeur, c'est un LLM : un moteur qui prédit du texte.
- ✓ Il peut se tromper avec assurance : on appelle ça une hallucination .
- ✓ Il est très fort pour transformer et expliquer , fragile sur les faits précis .
- ✓ OpenCode transforme ce moteur en assistant qui agit dans ton projet.

Le vocabulaire : modèle, provider, agent

« Un alphabet doré qui apprend à respirer. »

Six mots reviennent sans arrêt : LLM, modèle, provider, API, token, agent. Si tu les confonds, tout le reste paraît compliqué. Une fois clarifiés, la configuration d'OpenCode devient évidente.

AU PROGRAMME

1. Le LLM (le moteur)
2. Le modèle (la version précise)
3. Le token (l'unité de mesure)
4. La fenêtre de contexte (la mémoire de travail)
5. Le provider (le fournisseur)
6. L'API (le guichet)
7. L'agent (le chef d'orchestre)

À RETENIR

- ✓ Modèle = le moteur · Provider = le robinet.
- ✓ Token = l'unité de coût · Contexte = la mémoire de travail.
- ✓ API / clé = comment on y accède, et il faut la protéger.
- ✓ Agent = un LLM avec une mission et des outils : il agit.

Installer OpenCode (toutes les méthodes)

« Poser la première pierre d'une cité de lumière. »

OpenCode est un agent de code en ligne de commande, gratuit et open source. Il existe une méthode d'installation pour chaque système. On va choisir la tienne, l'installer, puis vérifier que tout fonctionne.

AU PROGRAMME

- Méthode 1 — L'installeur officiel (recommandé)
- Méthode 2 — Par gestionnaire de paquets
- Méthode 3 — Binaire autonome (Windows, ou sans gestionnaire)
- Méthode 4 — Docker, Desktop et Web
- Vérifier l'installation
- Désinstaller proprement

À RETENIR

- ✓ La méthode la plus simple : curl -fsSL https://opencode.ai/v2/install | bash .
- ✓ Sinon : gestionnaire de paquets, ou binaire téléchargeable (obligatoire sur Windows natif).
- ✓ On vérifie toujours avec opencode --version .
- ✓ La désinstallation ne supprime ni ta configuration ni tes sessions.

Premier lancement & l'interface

« Le premier souffle éveille la machine endormie. »

OpenCode se lance dans un dossier de projet . Il ouvre une interface plein écran (le TUI) et te laisse discuter avec l'IA. Voici tout ce qu'il faut savoir pour ne pas être perdu au premier lancement.

AU PROGRAMME

- Lancer OpenCode
- Les trois façons de l'utiliser
- L'écran du TUI, zone par zone
- Les 6 commandes à connaître absolument
- Le service en arrière-plan
- Questions fréquentes du premier lancement

À RETENIR

- ✓ On lance OpenCode dans le dossier du projet : opencode .
- ✓ Trois modes : opencode (TUI), run (script), mini (léger).
- ✓ /connect puis /models = les deux premières commandes à faire.
- ✓ Un service tourne en arrière-plan : on le gère avec opencode service .

Qu'est-ce qu'un provider ?

« Choisir la source d'où jaillit l'étincelle. »

Un provider, c'est le fournisseur qui met un modèle à ta disposition via Internet. Tu en relies un à OpenCode avec /connect , et c'est parti. Mais comprendre ce qui se passe derrière te fera économiser beaucoup de temps et d'argent.

AU PROGRAMME

- Provider, modèle, API : qui fait quoi
- La liste officielle des providers
- Connecter un provider
- Où sont mes clés ? (et pourquoi c'est sûr)
- Le cas particulier des variables d'environnement
- Combien de providers connecter ?

À RETENIR

- ✓ Un provider = un fournisseur de modèles accessible par API.
- ✓ On connecte avec /connect (TUI) ou opencode auth login (terminal).
- ✓ Les clés vivent dans la base locale d'OpenCode, pas en clair — mais on ne les partage jamais.
- ✓ Un ou deux providers suffisent pour commencer ; on en ajoute pour la résilience.

Trouver des modèles gratuits

« Les coffres ouverts du soleil, sans péage. »

On peut développer avec l'IA sans rien payer . La clé, c'est de savoir où sont les modèles gratuits, comment les identifier, et quelles sont leurs limites. Les listes changent souvent : on apprend donc surtout la méthode pour les trouver soi-même.

AU PROGRAMME

- OpenRouter : la source n°1 de modèles gratuits
- Hugging Face : des milliers de modèles open source
- Groq : la vitesse, avec un tier gratuit
- Google AI Studio (Gemini / Gemma)
- NVIDIA NIM — crédits offerts
- Le catalogue de référence : models.dev
- Mémo : où trouver du gratuit

À RETENIR

- ✓ Les modèles gratuits OpenRouter finissent par :free : filtre-les sur le site ou via l'API.
- ✓ Gratuit = limité : 20 req/min et 50 req/jour (1000 si tu as acheté ≥ 10 $).
- ✓ Alternatives gratuites : Hugging Face, Groq, Google AI Studio, NVIDIA NIM.
- ✓ La référence des catalogues, c'est models.dev .

Choisir son modèle (gratuit ou payant)

« Trier les étoiles selon leur juste éclat. »

Il existe des centaines de modèles. Bonne nouvelle : pour choisir, tu n'as besoin que de cinq critères . Et le meilleur modèle n'est presque jamais le plus « intelligent » : c'est celui qui répond au bon besoin au bon prix.

AU PROGRAMME

- Les 5 critères qui comptent
- Gratuit ou payant ? La règle simple
- Comprendre le prix (sans se ruiner)
- Options payantes utiles à connaître
- Identifier un bon modèle sur models.dev
- Les variants : plus ou moins de raisonnement

À RETENIR

- ✓ Cinq critères : contexte, outils, prix, qualité code, vitesse .
- ✓ Outils = obligatoire pour un agent ; sans tool calling, rien ne fonctionne.
- ✓ On paie par million de tokens , la sortie coûte plus cher que l'entrée.
- ✓ Un petit modèle pour le simple, un gros seulement quand c'est utile. Les variants aident.

Configurer ses providers (fichier de config)

« Accorder l'instrument avant le concert. »

Jusqu'ici on a tout fait à la souris. Maintenant on écrit le petit fichier qui rend ta configuration reproductible : modèle par défaut, providers, modèles sur mesure, variables d'environnement. C'est la pièce qui fait passer de « bricolage » à « poste de travail ».

AU PROGRAMME

- Où se trouve ce fichier ?
- Un fichier minimal
- Un fichier complet, commenté
- Rediriger un provider existant
- Variants et coûts d'un modèle
- Vérifier que la config est prise en compte

À RETENIR

- ✓ Global : ~/.config/opencode/opencode.jsonc ; projet : opencode.jsonc .
- ✓ V2 utilise le pluriel : providers , agents , plugins .
- ✓ Les secrets passent par {env:NOM} , jamais en clair.
- ✓ Un provider personnalisé se décrit avec package + settings.baseURL + models .

Modèles locaux (Ollama, LM Studio, vLLM)

« Une flamme qui brûle sur ta propre table. »

Et si le modèle tournait sur ta propre machine ? Zéro clé API, zéro facture, zéro Internet. C'est plus lent et moins puissant qu'un gros modèle cloud, mais pour beaucoup de tâches c'est largement suffisant — et parfait pour les données sensibles.

AU PROGRAMME

- Comment ça marche
- Installer Ollama
- Laisser OpenCode découvrir Ollama
- LM Studio et vLLM

À RETENIR

- ✓ Ollama / LM Studio / vLLM = modèles sur ta machine , sans clé ni facture.
- ✓ OpenCode les détecte automatiquement sur leurs adresses locales.
- ✓ Choisis la taille du modèle selon ta RAM/VRAM.
- ✓ Vérifie toujours le support des outils : sans lui, pas d'agent.

Réduire les coûts (et rester efficace)

« Dompter la rivière d'or qui fuit entre les doigts. »

Beaucoup pensent que l'IA coûte cher. En réalité, mal configurée, elle coûte cher ; bien configurée, elle coûte des centimes. Voici les leviers concrets, du plus efficace au plus fin.

AU PROGRAMME

- Levier 1 — Un petit modèle pour les petites tâches
- Levier 2 — Le bon variant au bon moment
- Levier 3 — Réduire le contexte (le plus gros levier)
- Levier 4 — La compaction automatique
- Levier 5 — Limiter ce que l'agent peut faire
- Levier 6 — Les tokens en cache
- Levier 7 — Le basculement automatique (fallback)

À RETENIR

- ✓ Le plus gros levier : réduire le contexte envoyé.
- ✓ Petit modèle pour le simple, variant #low pour le mécanique.
- ✓ La compaction résume automatiquement le vieux contexte.
- ✓ Le fallback (évite les blocages de quota) se configure côté provider ou via plugin.

Qu'est-ce qu'un agent ?

« Un automate doré qui rêve d'agir. »

Jusqu'ici, tu as parlé à un modèle. Un agent va plus loin : il a un rôle, des outils, des règles, et parfois même des agents assistants. C'est la brique qui fait vraiment gagner du temps.

AU PROGRAMME

- La recette d'un agent
- Les trois modes
- Les agents intégrés à OpenCode
- Le bon réflexe : plan avant build
- Les permissions : le contrôle

À RETENIR

- ✓ Un agent = rôle + modèle + outils + permissions + limites .
- ✓ Trois modes : primary , subagent , all .
- ✓ Intégrés : build , plan , general , explore .
- ✓ Les permissions limitent chaque agent : dernier règle gagnante.

Créer un agent local (pas à pas)

« Façonner une silhouette de lumière sur mesure. »

Un agent, c'est juste un fichier texte dans un dossier. On va en créer un vrai, du nom à la permission, puis l'utiliser.

AU PROGRAMME

- Où placer le fichier ?
- Pas à pas
- Faire de l'agent le principal (default_agent)
- La variante JSON (même agent, dans la config)
- Trois idées d'agents à recopier

À RETENIR

- ✓ Un agent = un fichier .md dans ~/.config/opencode/agents/ ou .opencode/agents/ .
- ✓ Le frontmatter définit description , mode , model , permissions ; le corps Markdown devient le prompt système.
- ✓ default_agent choisit l'agent de départ.
- ✓ Même agent définissable en JSON sous agents .

Trouver & intégrer des agents

« Fouiller le grand marché des ombres utiles. »

Pas besoin de tout écrire soi-même. La communauté partage des agents prêts à l'emploi. Voici où les trouver, comment les intégrer proprement, et surtout comment vérifier ce que tu installes.

AU PROGRAMME

- Où trouver des agents
- Intégrer un agent en 4 étapes
- Adapter un agent à ton projet

À RETENIR

- ✓ On trouve des agents en ligne (GitHub, communautés), mais le meilleur est souvent le tien.
- ✓ Installation : déposer un .md dans agents/ .
- ✓ Toujours lire avant d'installer : un agent = du code avec des droits.
- ✓ Adapter le modèle et le contexte rend un agent générique vraiment utile.

Les Skills : des compétences réutilisables

« Offrir à la machine de nouveaux gestes appris. »

Tu répètes souvent les mêmes consignes à l'agent ? Une skill (compétence) est une fiche de consignes rangée à part, qu'OpenCode va chercher tout seul quand le sujet s'y prête. Tes prompts restent courts, tes consignes ne se perdent plus.

AU PROGRAMME

- Le problème qu'elles résolvent
- Anatomie d'une skill
- Un exemple complet
- Où OpenCode les cherche (automatiquement)
- L'identifiant (ID), une règle à retenir
- Qui gagne en cas de doublon ?
- Ajouter d'autres sources (locales ou HTTP)

À RETENIR

- ✓ Une skill = un dossier avec SKILL.md (ou un .md racine).
- ✓ La description sert à la découvrir automatiquement .
- ✓ L' ID vient du chemin , pas du name ; il est sensible à la casse.
- ✓ Chargement à la demande : @id , ou via l'outil skill .
- ✓ Permission skill pour autoriser, demander ou bloquer.

Les plugins : brancher des outils

« Brancher des membres supplémentaires au colosse. »

Un plugin ajoute des outils , des hooks et des intégrations à OpenCode. C'est la manière la plus puissante d'étendre l'assistant : on branche un paquet npm ou un petit fichier TypeScript, et OpenCode gagne une nouvelle capacité.

AU PROGRAMME

- Plugin ou skill ?
- Installer un plugin avec la CLI
- Ce que la CLI écrit dans la config
- Découverte automatique
- Activer, désactiver, ordonner
- Plugins de terminal (TUI)
- Recharger après un changement

À RETENIR

- ✓ Un plugin = du code qui ajoute des capacités (outils, hooks, intégrations).
- ✓ Installation CLI : opencode plugin add / list / update / remove .
- ✓ Découverte auto dans .opencode/plugins/ et ~/.config/opencode/plugins/ .
- ✓ - désactive, * vise tout, .* un préfixe.
- ✓ Plugins TUI à part, dans cli.json .

Les commandes : tes raccourcis

« Murmurer des ordres brefs à l'oreille du sphinx. »

Une commande transforme un prompt que tu répètes en un simple /mot . Tu écris la consigne une fois dans un fichier, et tu la déclenches en une frappe. Idéal pour standardiser une façon de travailler.

AU PROGRAMME

- La commande la plus simple
- Où ranger ses commandes
- Les arguments : $ARGUMENTS, $1, $2…
- Insérer une commande shell
- Choisir l'agent, le modèle, l'exécution
- La forme JSON

À RETENIR

- ✓ Une commande = un fichier .md dans .opencode/commands/ ou ~/.config/opencode/commands/ .
- ✓ On déclenche avec /nom ; le sous-dossier devient un préfixe.
- ✓ Arguments : $ARGUMENTS (tout), $1 , $2 … (positionnels).
- ✓ Bloc !`cmd` : injecte une sortie shell (attention, exécution immédiate).
- ✓ Frontmatter : agent , model , subagent .

Les serveurs MCP : connecter le monde

« Tisser des ponts entre les îles de données. »

MCP (Model Context Protocol) est un standard qui permet à OpenCode de parler à des services externes : une base de données, une API, un CRM, la documentation d'une librairie. C'est la pièce qui transforme l'assistant en outil branché sur ton système d'information.

AU PROGRAMME

- Distant ou local ?
- Ajouter un serveur avec la CLI
- Authentification (OAuth)
- La configuration à la main
- Comment les outils s'appellent
- Limiter et couper un serveur
- Brancher un outil métier (ex. un CRM)

À RETENIR

- ✓ MCP = standard pour brancher des outils externes à l'agent.
- ✓ Deux types : remote (URL) et local (commande stdio).
- ✓ CLI : opencode mcp add / list / auth / logout ; dans l'interface : /mcps .
- ✓ Config sous mcp.servers (jamais directement sous mcp).
- ✓ Noms d'outils normalisés <serveur>_<outil> ; permissions pour bloquer.

Bien prompter & le fichier AGENTS.md

« L'incantation qui ordonne le chaos en dialogue. »

Un bon agent avec un mauvais prompt donne de mauvais résultats. La compétence la plus rentable n'est pas technique : c'est de savoir demander . Et il existe un fichier magique qui évite de tout répéter : AGENTS.md .

AU PROGRAMME

- La recette d'un bon prompt
- Les 5 réflexes qui changent tout
- AGENTS.md : le contexte permanent

À RETENIR

- ✓ Un bon prompt = contexte + objectif + contraintes + format + demande de plan .
- ✓ Planifier avant d'agir économise du temps et des tokens.
- ✓ AGENTS.md porte le contexte du projet ; /init le génère.
- ✓ Court et concret : des règles précises sont mieux suivies.

Mini-projet : ton assistant de A à Z

« Élever, pierre à pierre, ta propre cathédrale de code. »

Place à la pratique. On construit, étape par étape, un assistant de projet complet : contexte, agent de revue, skill, commande et serveur MCP. Chaque étape réutilise ce que tu as appris — rien de nouveau.

AU PROGRAMME

- Ce qu'on va construire
- Étape 1 — Le contexte du projet
- Étape 2 — Un agent de revue
- Étape 3 — Une skill réutilisable
- Étape 4 — Une commande
- Étape 5 — Un serveur MCP
- Étape 6 — Le test final

À RETENIR

- ✓ AGENTS.md = contexte ; l'agent le lit tout seul.
- ✓ Le subagent de revue illustre le moindre privilège (lecture seule).
- ✓ Skill = méthode réutilisable ; commande = raccourci ; MCP = outil externe.
- ✓ Une config d'assistant se versionne et se partage avec l'équipe.

Synthèse, dépannage & ressources

« Le miroir universel interroge le soleil levant. »

Félicitations : tu as fait tout le parcours. Récapitulons l'essentiel, voyons comment résoudre les problèmes courant, et où continuer en autonomie.

AU PROGRAMME

- Le récapitulatif du cours
- Dépannage rapide
- Les commandes à retenir
- Ressources fiables

À RETENIR

- ✓ Le trio gagnant : bon modèle + bon contexte + bon prompt .
- ✓ Étends OpenCode seulement quand tu en as besoin (skills, plugins, MCP).
- ✓ Moindre privilège partout : un agent = le minimum de droits.
- ✓ Vérifie toujours la documentation officielle avant d'appliquer une astuce.
- ✓ Tu sais maintenant installer, configurer et étendre ton assistant de code.

FIN DU PARCOURS

Tu as maintenant ton assistant de code

Le cours complet, les commandes d'installation et tous les exemples de code se trouvent dans index.html du projet. Ouvre-le dans un navigateur : 20 chapitres, une tuile par thématique.

1. Comprendre l'IA
2. Installer OpenCode
3. Choisir ses modèles
4. Créer ses agents
5. Étendre : skills, plugins, MCP
6. Prompter & maîtriser les coûts